



05506026

Digital Image Processing

Witchaya Towongpaichayont
Computer Science, KMITL

CONTENTS



1

IMAGE RESTORATION (1)



IMAGE RESTORATION

Image Degradation and Restoration

- ▶ When an image $f(x, y)$ is degraded (processed into less quality) into a degraded image $g(x, y)$ by operation $\mathcal{H}$ plus noise η . By having some knowledge about $\mathcal{H}$ and η , it might be possible to restore the estimation of original image $\hat{f}(x, y)$

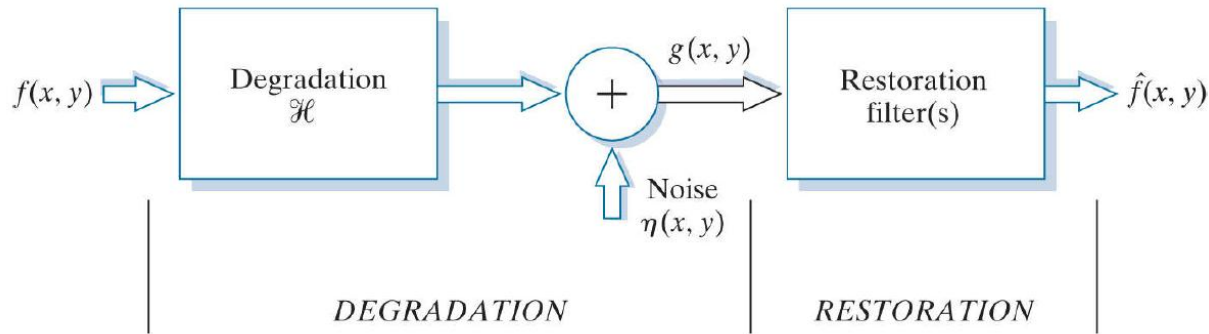


Image Degradation and Restoration

- ▶ $f(x, y)$ – image before degradation, ‘true image’
- ▶ $g(x, y)$ – image after degradation, ‘observed image’
- ▶ $h(x, y)$ – degradation filter
- ▶ $\hat{f}(x, y)$ – estimate of $f(x, y)$ computed from $g(x, y)$
- ▶ $\eta(x, y)$ – additive noise

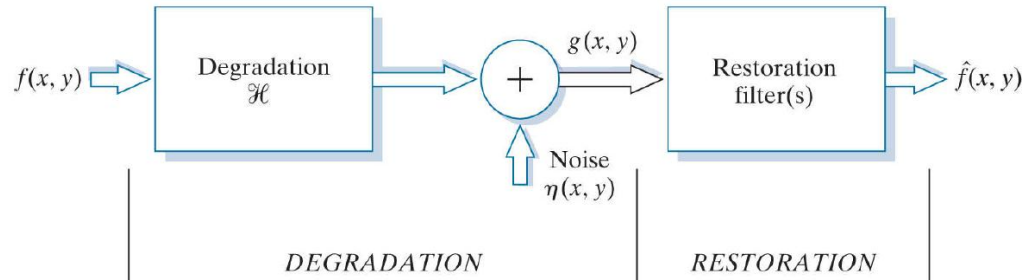


Image Degradation and Restoration



- original



- optical blur



- motion blur



- spatial quantization (discrete pixels)



- additive intensity noise

Image Degradation and Restoration

- ▶ If $\mathcal{H}$ is a linear, position-invariant operator, the degraded image in the spatial domain is

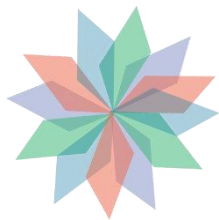
$$g(x, y) = (h \star f)(x, y) + \eta(x, y)$$

- ▶ Where $\star$ indicated convolution and $h(x, y)$ is the spatial representation of the degradation function
- ▶ On the other hand, the degraded image in the frequency domain is

$$G(u, v) = H(u, v)F(u, v) + N(u, v)$$



Noise Reduction

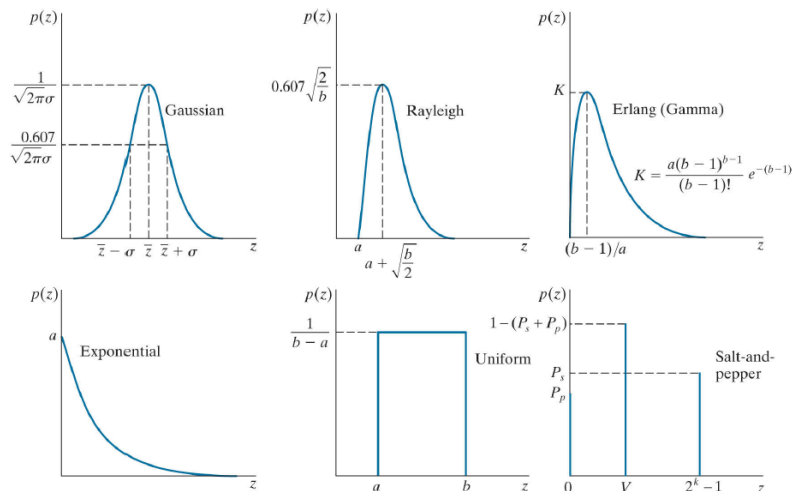


Noise Models

- ▶ The principal sources of noise in digital images arise during image acquisition and/or transmission
- ▶ The performance of imaging sensors is affected by a variety of environmental factors during image acquisition, and by the quality of the sensing elements themselves

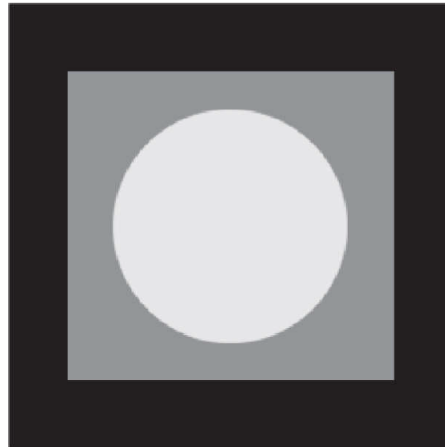
Probability Density Function

- Probability Density Function (PDF) is a function of each noise model which indicates the chance of noise within an image. The PDF of some noise models are as below



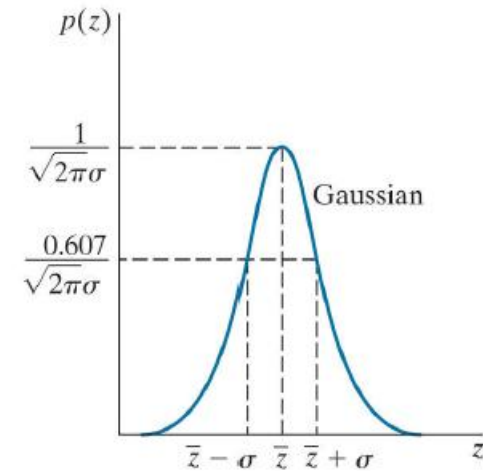
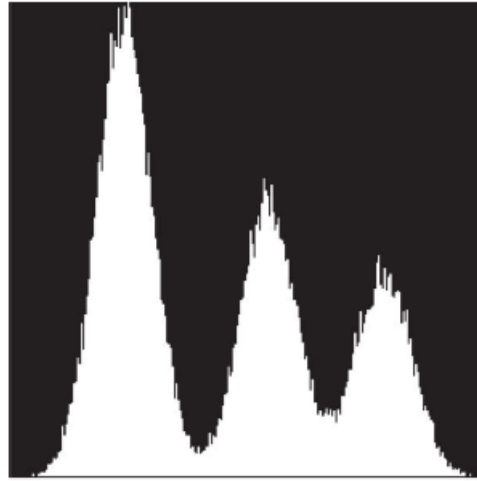
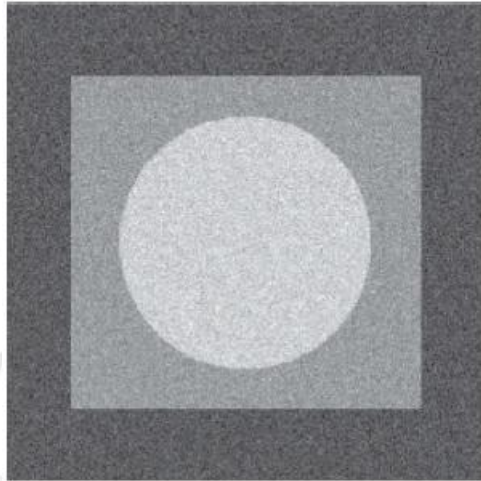
Noisy images and their histograms

- ▶ We will now test the image below with different models of noise and compare their histograms and their PDF function



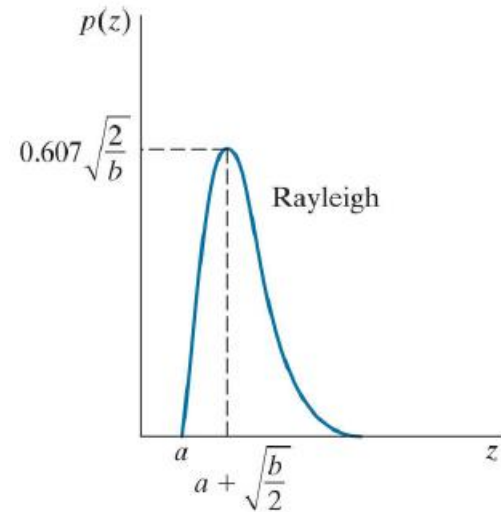
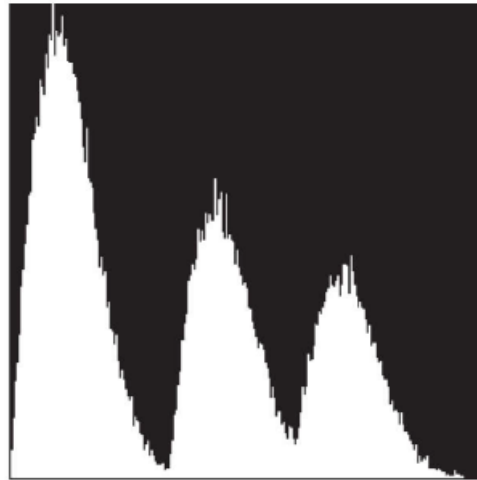
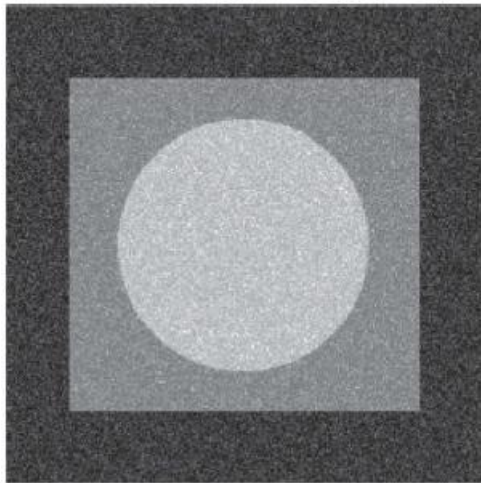
Noisy images and their histograms

► Gaussian noise



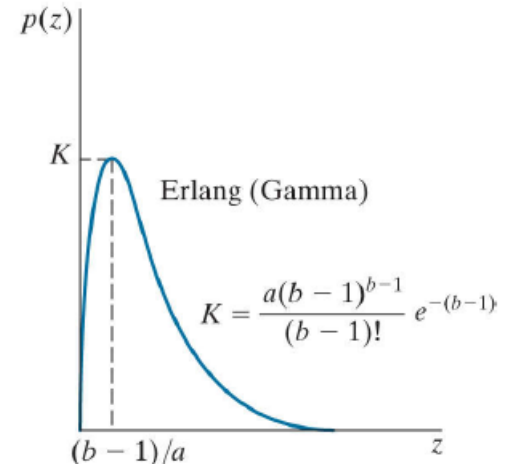
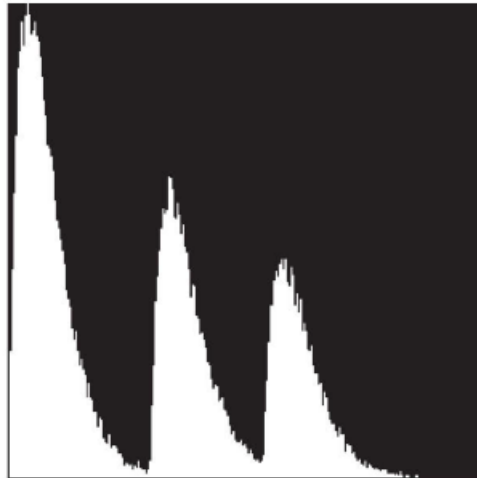
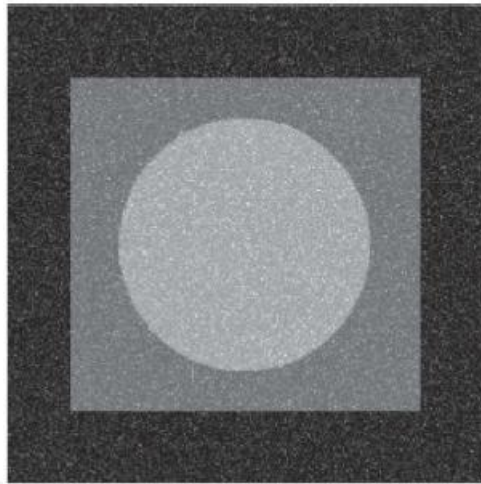
Noisy images and their histograms

► Rayleigh noise



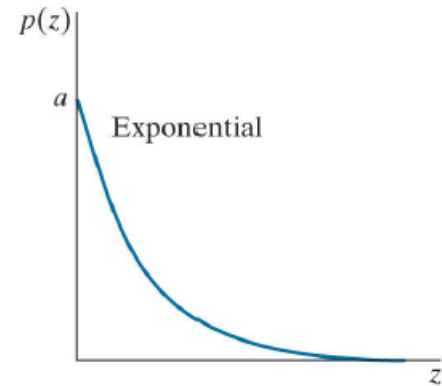
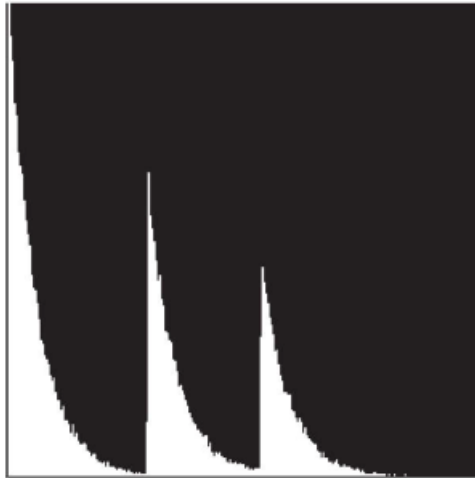
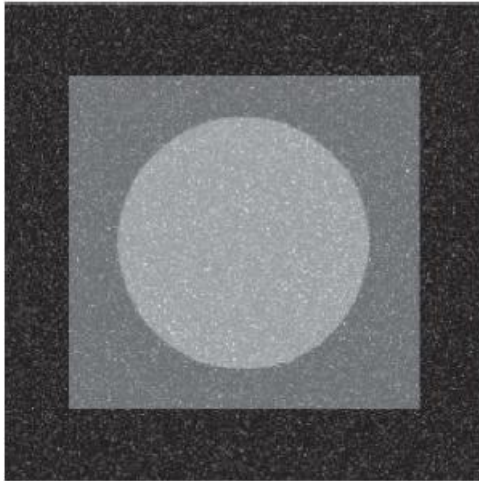
Noisy images and their histograms

► Erlanga noise



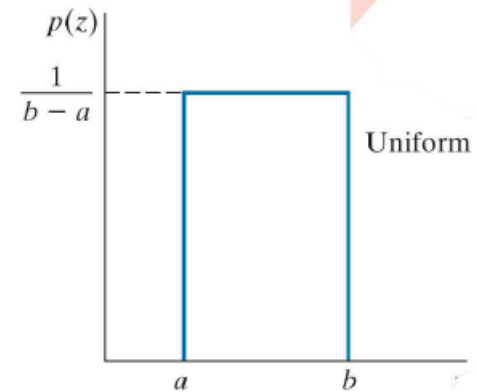
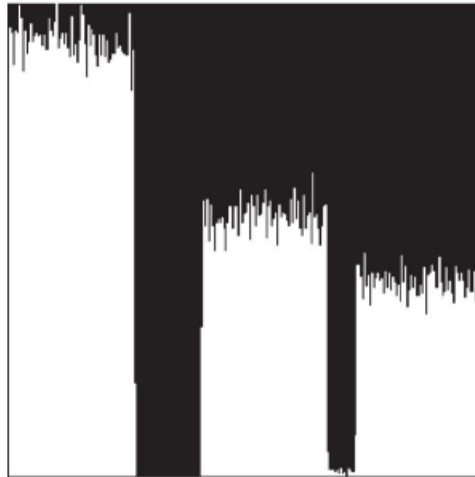
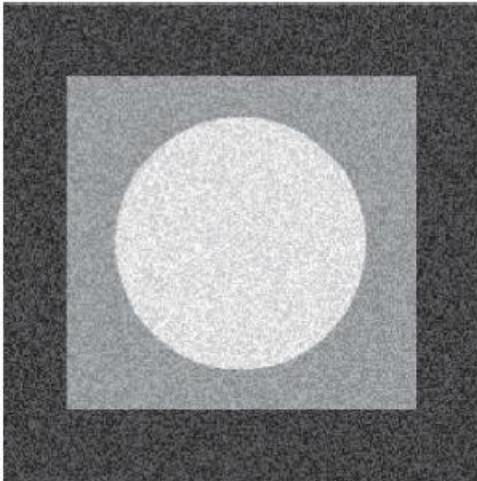
Noisy images and their histograms

► Exponential noise



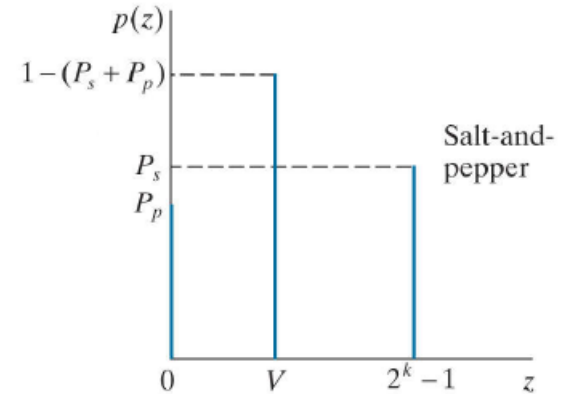
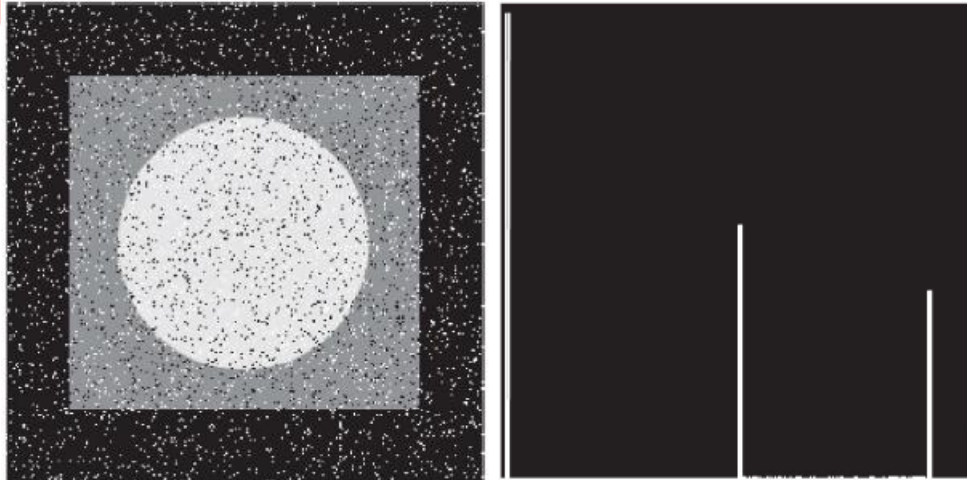
Noisy images and their histograms

► Uniform noise



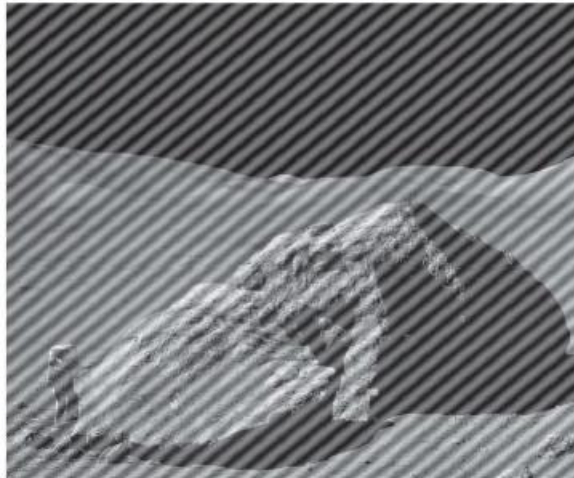
Noisy images and their histograms

► Salt-and-pepper noise



Periodic Noise

- ▶ Periodic noise typically arises from electrical or electromechanical interference during image acquisition.





Noise Reduction – Spatial Filtering

Denoising image

- ▶ If we consider only the presence of noise, an image is degraded as below

$$g(x, y) = f(x, y) + \eta(x, y)$$

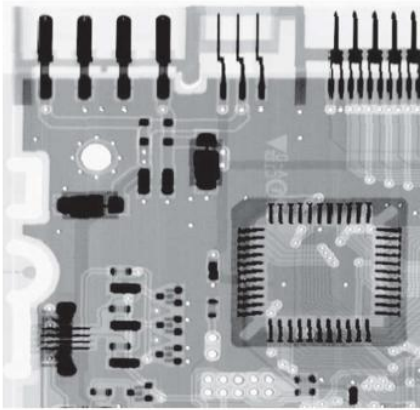
$$G(u, v) = F(u, v) + N(u, v)$$

- ▶ In order to denoise, we can use spatial filtering when only random noise is present

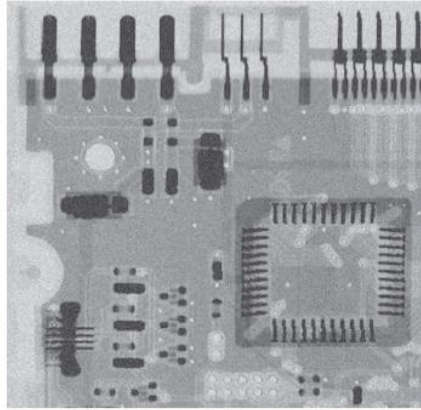
Mean Filters - Arithmetic

Simplest Mean Filter

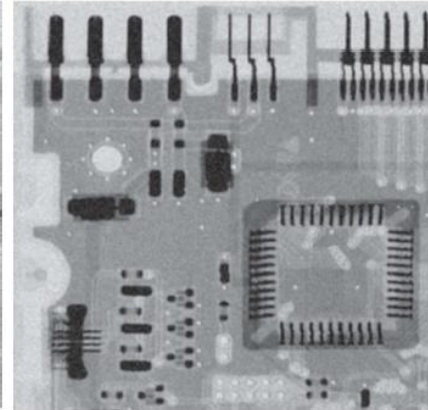
$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(r, c) \in S_{xy}} g(r, c)$$



Original Image



**Corrupted with
Gaussian noise**

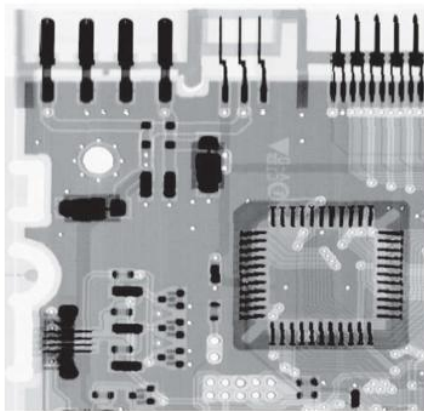


**Filtered with
Arithmetic Mean Filter**

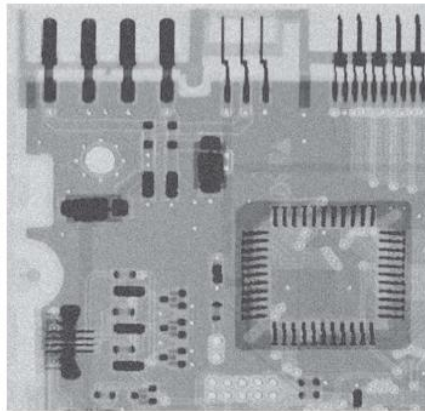
Mean Filters - Geometric

Same level of smoothing
result as arithmetic, better
in losing detail

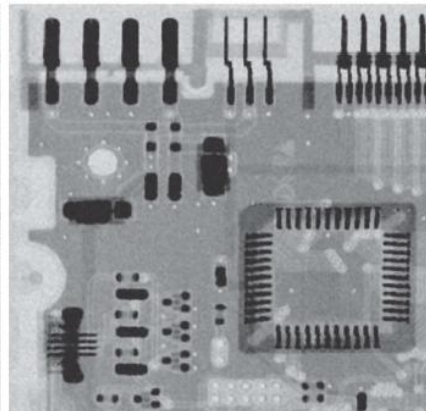
$$\hat{f}(x,y) = \left[\prod_{(r,c) \in S_{xy}} g(r,c) \right]^{\frac{1}{mn}}$$



Original Image



**Corrupted with
Gaussian noise**



**Filtered with
Geometric Mean Filter**

Mean Filters - Harmonic

Works well for salt noise, fails for pepper noise.

$$\hat{f}(x, y) = \frac{mn}{\sum_{(r, c) \in S_{xy}} \frac{1}{g(r, c)}}$$

Mean Filters - Contraharmonic

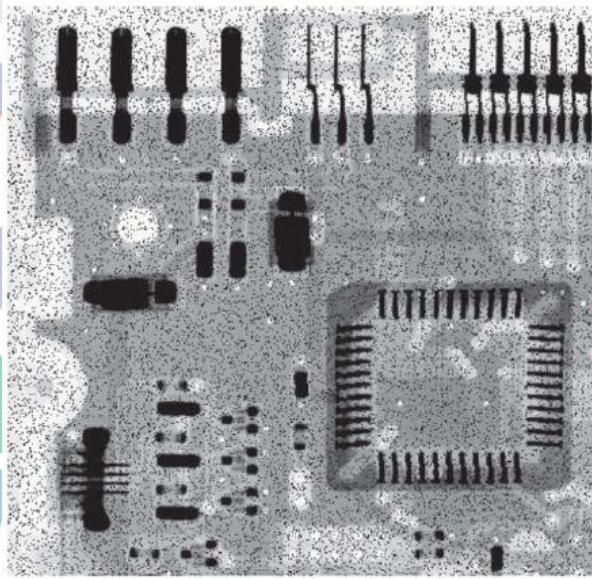
Q is the order of the filter.

Positive Q for pepper noise

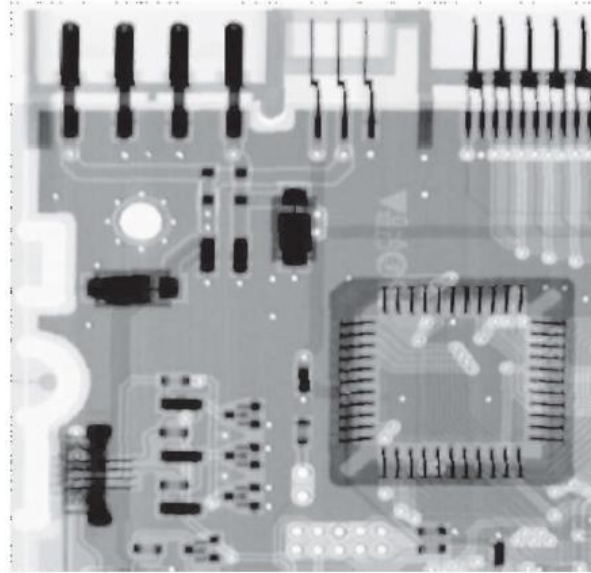
Negative Q for salt noise

Cannot do both simultaneously

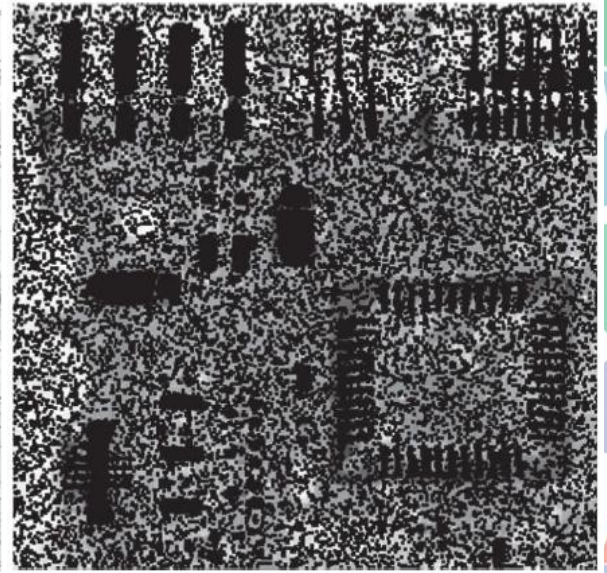
$$\hat{f}(x, y) = \frac{\sum_{(r, c) \in S_{xy}} g(r, c)^{Q+1}}{\sum_{(r, c) \in S_{xy}} g(r, c)^Q}$$



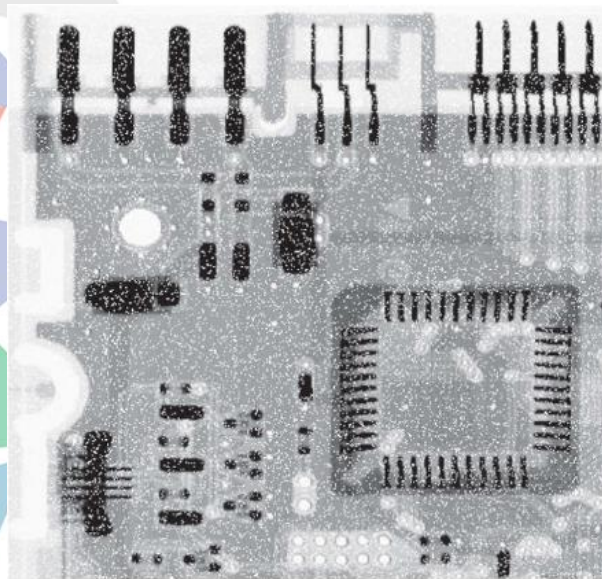
**Pepper Noise
With $p = 0.1$**



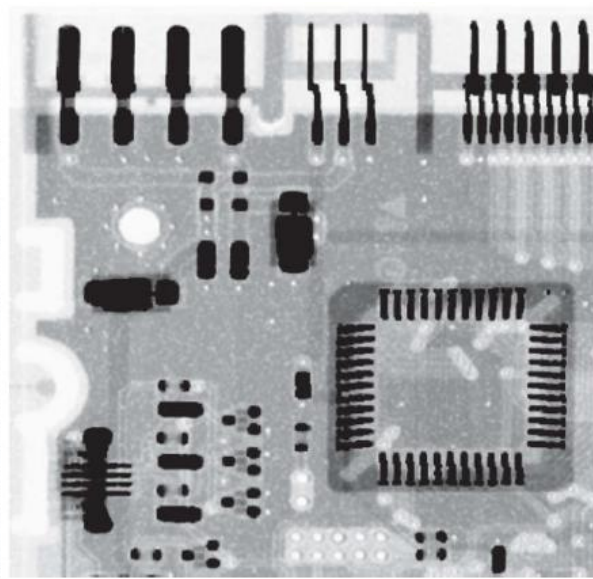
**Filtered By
Contraharmonic filter
 $Q = 1.5$**



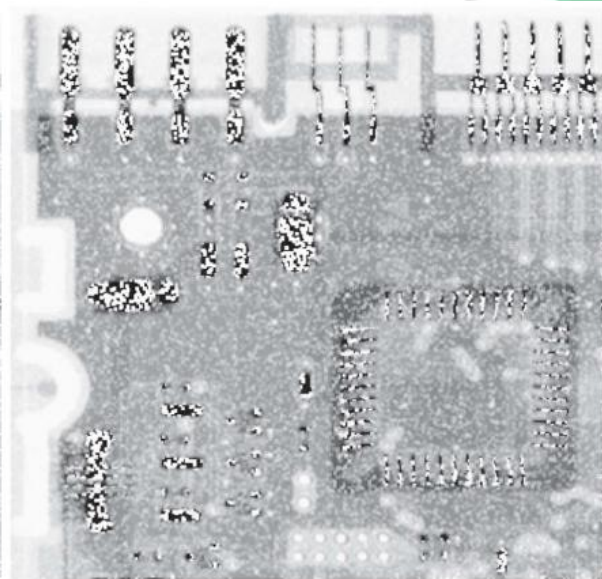
**Filtered By
Contraharmonic filter
 $Q = -1.5$**



**Salt Noise
With $p = 0.1$**



**Filtered By
Contraharmonic filter
 $Q = -1.5$**

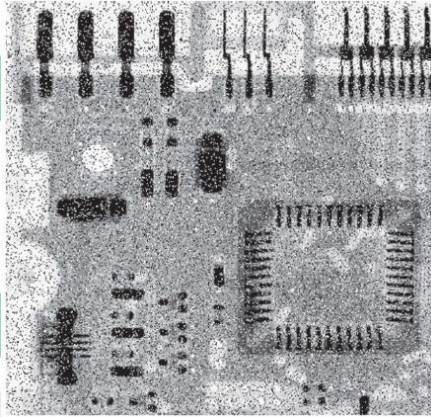


**Filtered By
Contraharmonic filter
 $Q = 1.5$**

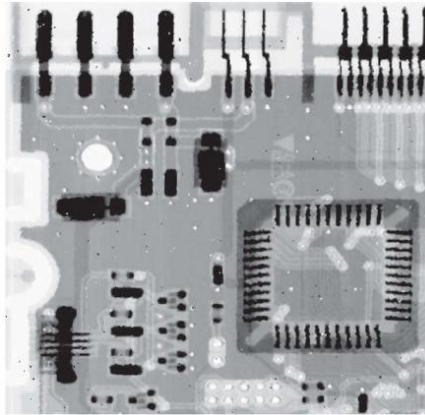
Order-Statistic Filters - Median

Simplest Order-Statistic Filter

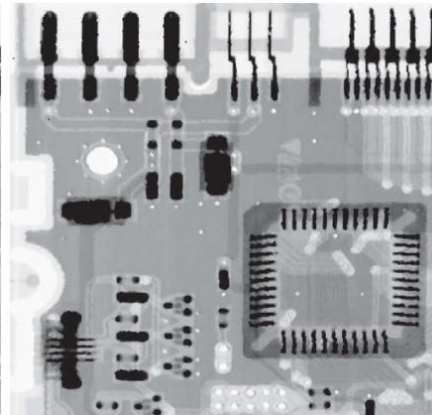
$$\hat{f}(x,y) = \underset{(r,c) \in S_{xy}}{\text{median}} \{g(r,c)\}$$



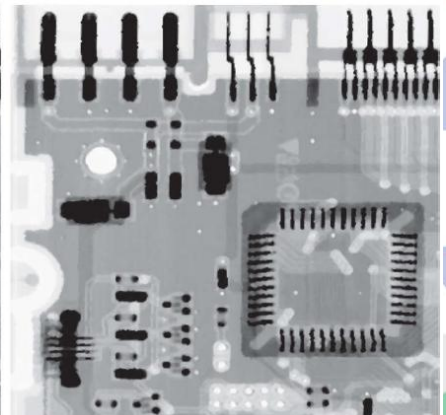
Salt and pepper Noise
With $p_s = p_p = 0.1$



1st pass
3x3 Median filter



2nd pass
3x3 Median filter



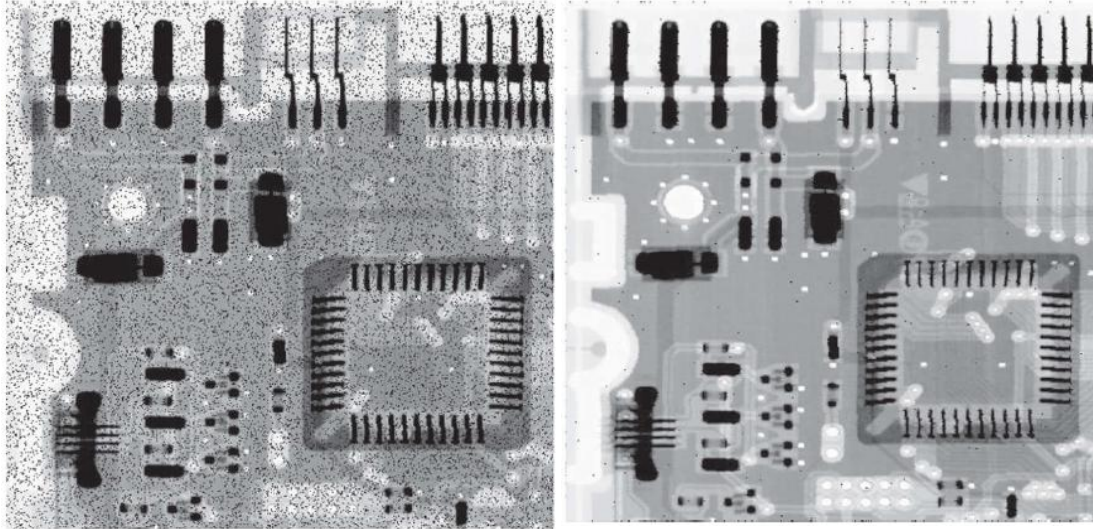
3rd pass
3x3 Median filter

Order-Statistic Filters – Max and Min

Max is for pepper noise

$$\hat{f}(x,y) = \max_{(r,c) \in S_{xy}} \{g(r,c)\}$$

Pepper Noise
With $p = 0.1$



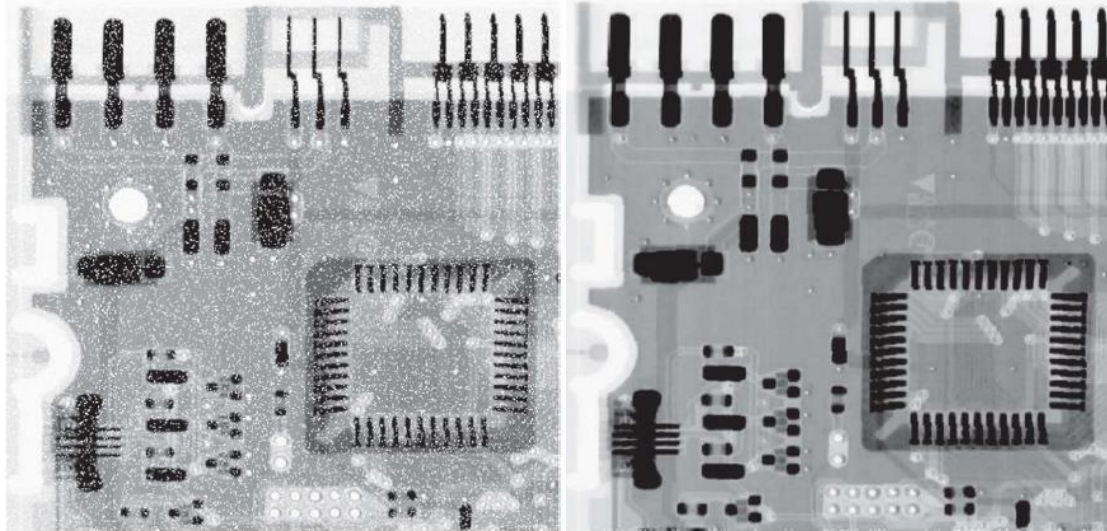
Filtered by
3x3 Max filter

Order-Statistic Filters – Max and Min

Min is for salt noise

$$\hat{f}(x,y) = \min_{(r,c) \in S_{xy}} \{g(r,c)\}$$

Salt Noise
With $p = 0.1$



Filtered by
3x3 Min filter

Order-Statistic Filters – Midpoint

Works well for randomly distributed noise – Gaussian, Uniform

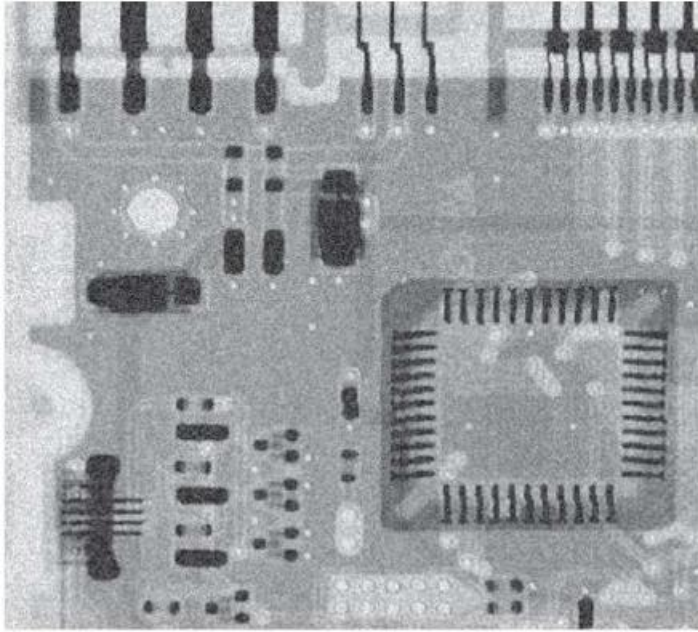
$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(r, c) \in S_{xy}} \{g(r, c)\} + \min_{(r, c) \in S_{xy}} \{g(r, c)\} \right]$$

Order-Statistic Filters – Alpha-Trimmed Mean

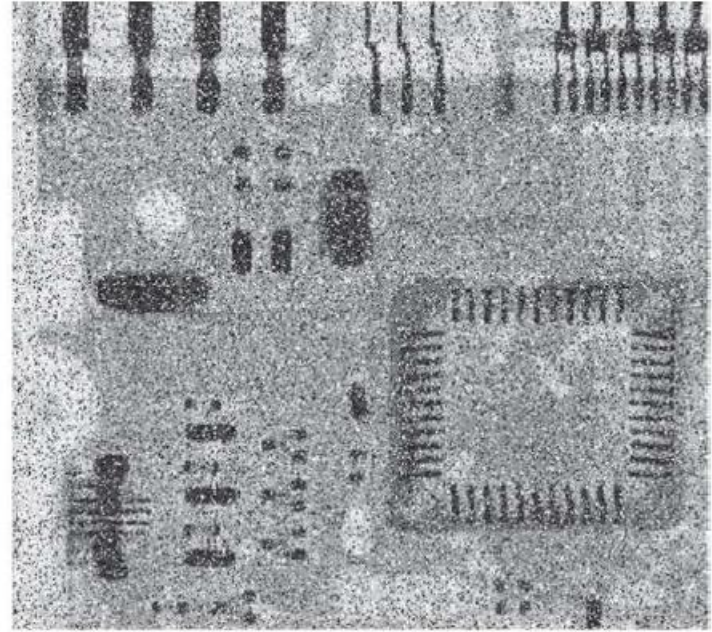
d is the number of values which will be deleted ($d/2$ lowest and $d/2$ highest intensity values of $g(r,c)$). After that, the remaining values will be averaged

$$\hat{f}(x,y) = \frac{1}{mn - d} \sum_{(r,c) \in S_{xy}} g_R(r,c)$$

Good for situations where there are multiple combined types of noise

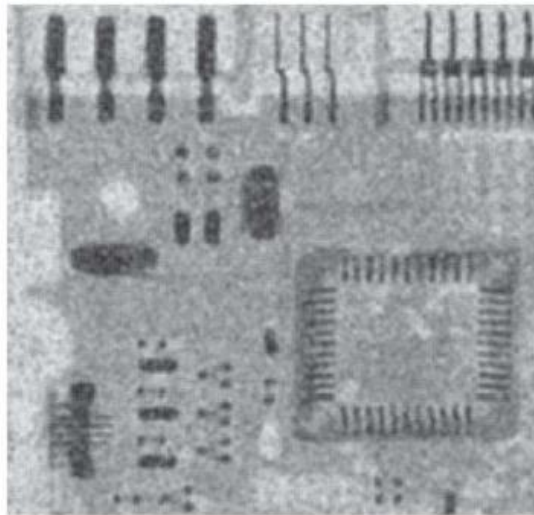


**Uniform noise
added**

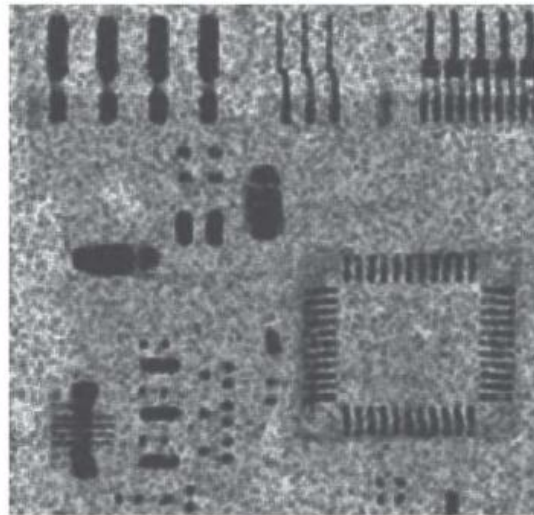


**Salt-and-pepper noise
added**

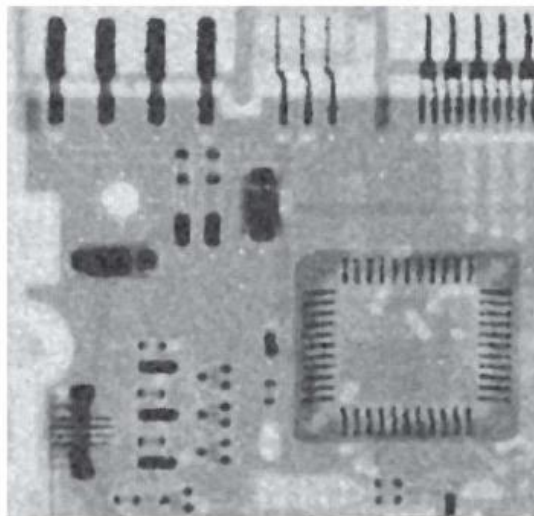
**Arithmetic mean
filter**



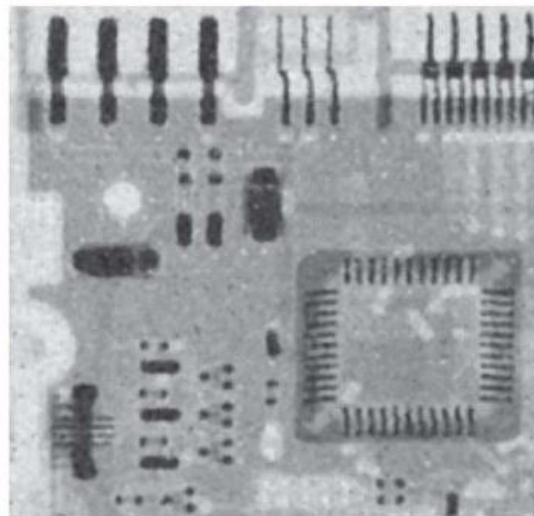
**Geometric mean
filter**



Median filter



**Alpha-trimmed
Mean filter**

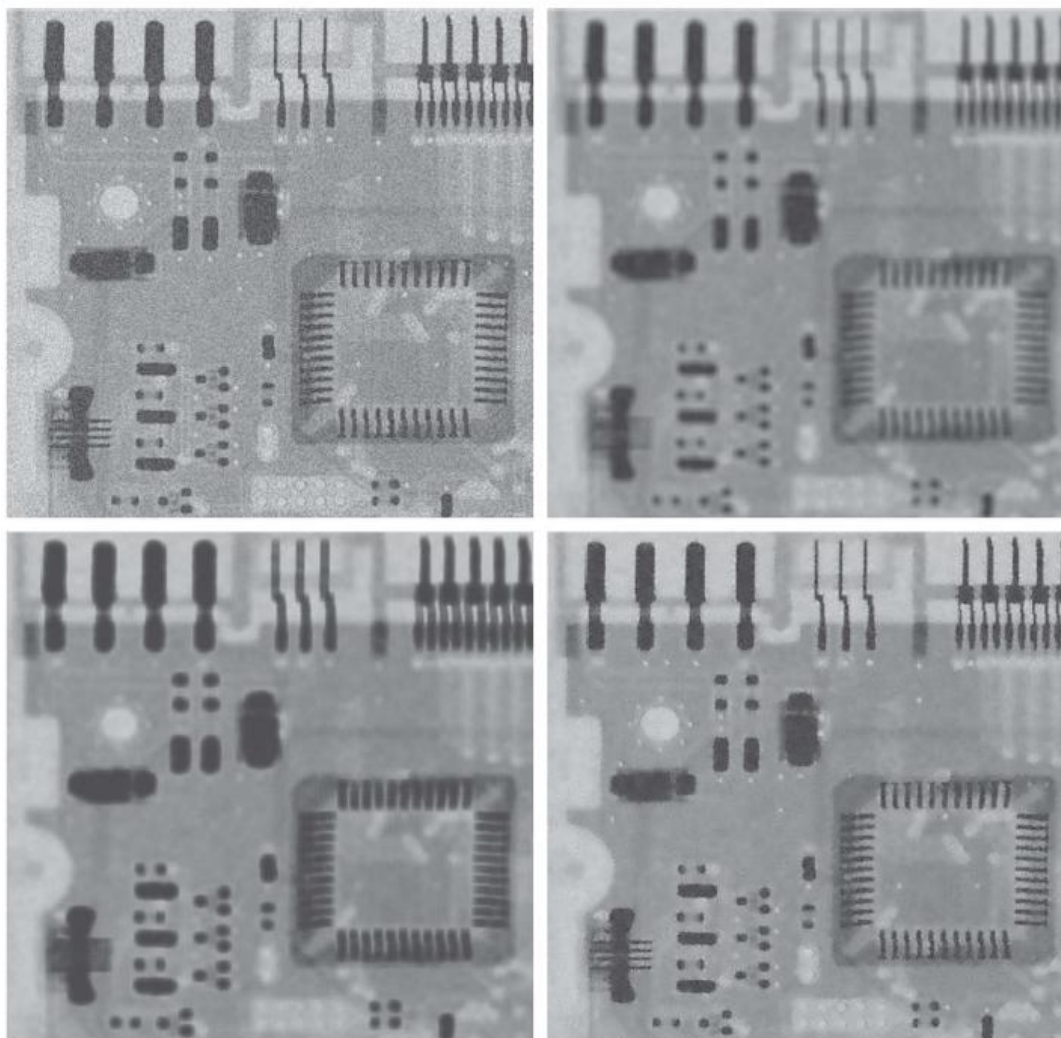


Adaptive Filter

Superior in result, inferior in computing cost

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_{\eta}^2}{\sigma_{S_{xy}}^2} [g(x, y) - \bar{z}_{S_{xy}}]$$

**Corrupted by
Gaussian noise**



**Geometric mean
filter**

**Arithmetic mean
filter**

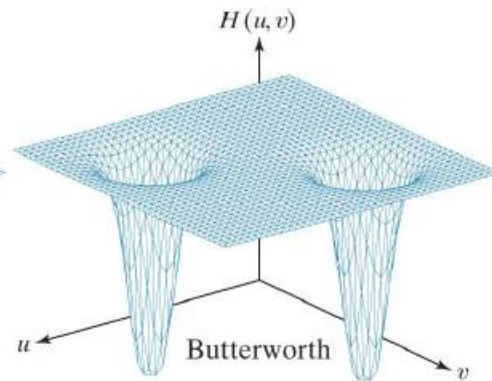
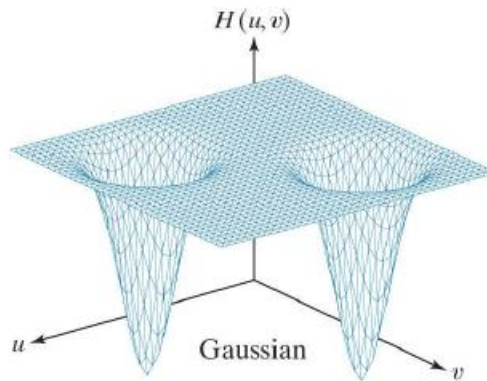
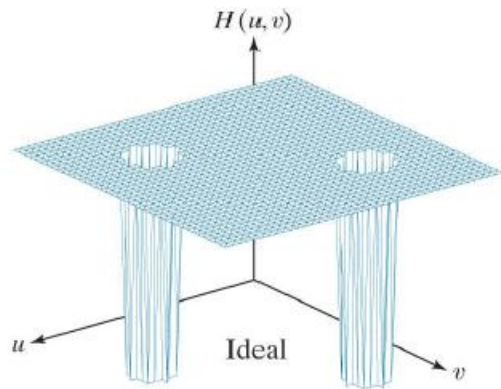
Adaptive filter



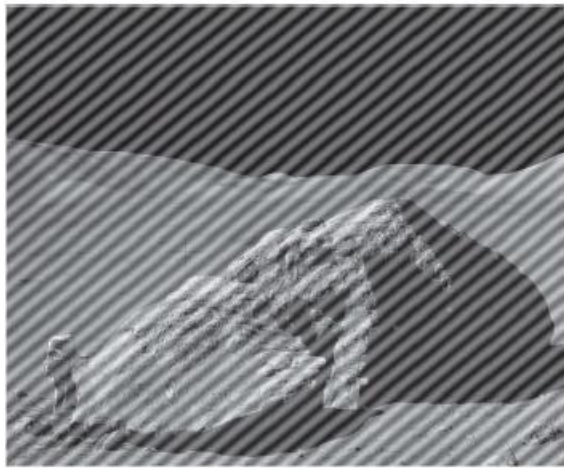
Noise Reduction – Frequency Domain Filtering

Notch Filter

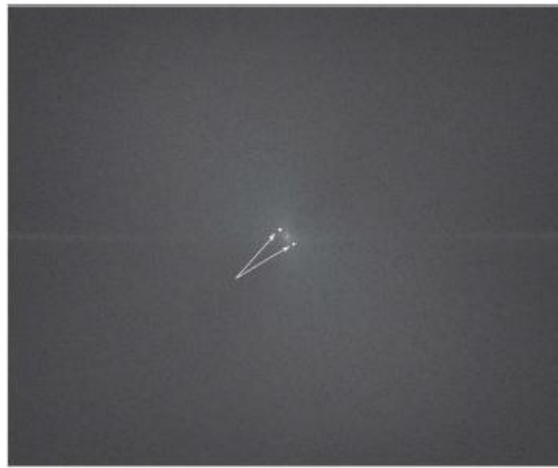
$$H_{\text{NR}}(u, v) = \prod_{k=1}^Q H_k(u, v) H_{-k}(u, v)$$



**Corrupted by
sinusoidal noise**



**Spectrum in
frequency domain**



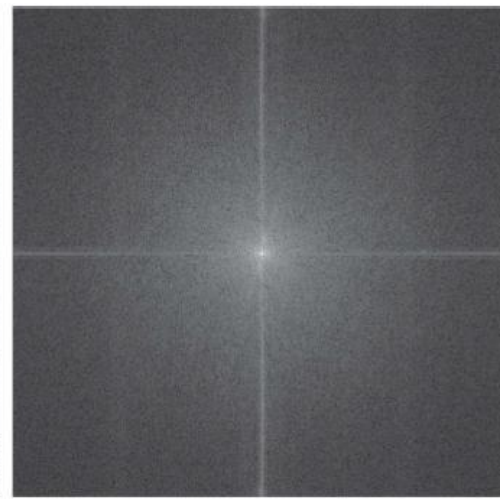
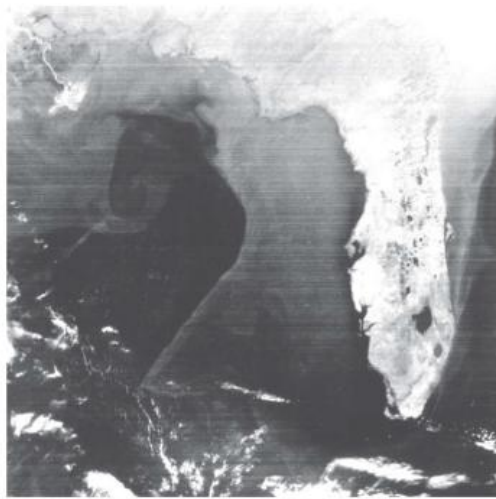
Notch filter



After filtered

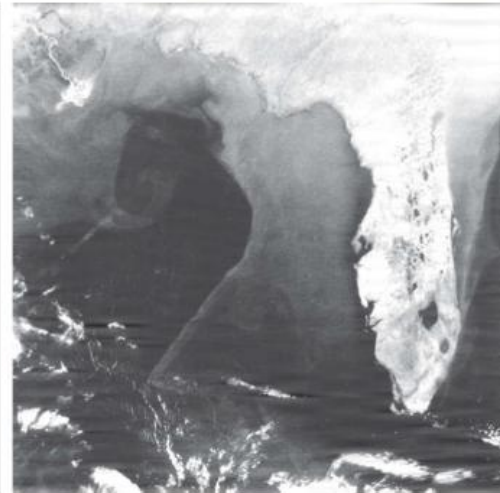
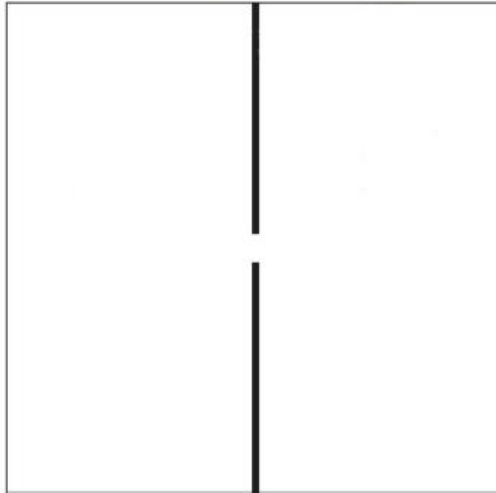


**Corrupted by
sinusoidal noise**



**Spectrum in
frequency domain**

Notch filter

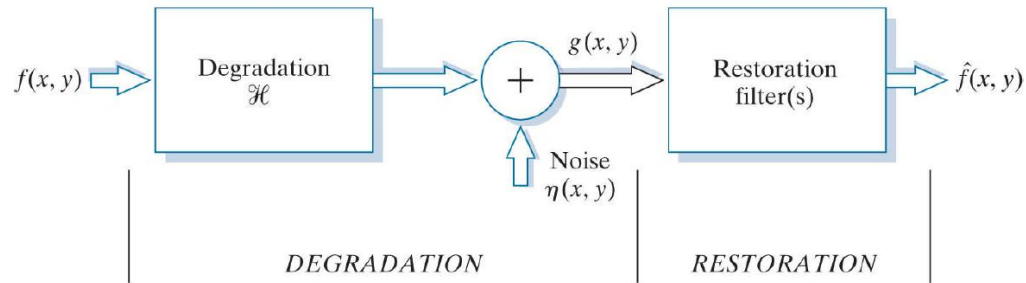


After filtered



Restore Degradation Function

Degradation function



- ▶ Back to this chart, the degradation function is $h(x, y)$ which produces $g(x, y)$ by operating on the image $f(x, y)$

$$g(x, y) = (h \star f)(x, y) + \eta(x, y)$$

$$G(u, v) = H(u, v)F(u, v) + N(u, v)$$

- ▶ Therefore, in order to restore the image, this function need to be inversed

Estimating the degradation function

The problem is, normally we do not know the degradation function. So, we need to estimate

There are 3 principle ways to estimate the degradation function

- **Observation** – observe through a small section of the image
- **Experimentation** – If the degradation is occurred by an equipment, it might be possible to reproduce the degradation by experimenting various settings on the equipment
- **Modeling**

Inverse Filtering

$$G(u, v) = H(u, v)F(u, v) + N(u, v)$$

- ▶ As the forward operation in frequency domain is as above, the inverse operation is

$$\hat{F}(u, v) = F(u, v) + \frac{N(u, v)}{H(u, v)}$$

- ▶ Therefore, the operation is to convert the image into Fourier Series -> inverse the function -> convert back to spatial domain

Example

$$h(x,y) = \frac{1}{2\pi\sigma^2} e^{-r^2/2\sigma^2}$$

$n(x,y)$ = Normal distribution, mean zero



$f(x,y)$



$g(x,y)$

blur $\sigma = 1.0$ pixels
noise $\sigma = 0.3$ grey levels

noise $\sigma = 0.3$ grey levels

$$\hat{F}(u,v) = G(u,v) / H(u,v)$$

blur $\sigma = 0.5$ pixels



$g(x,y)$

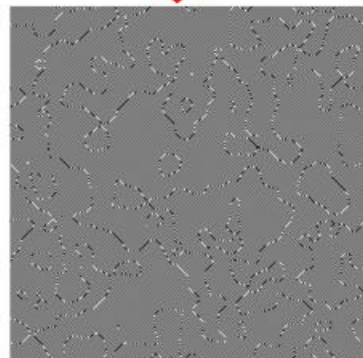
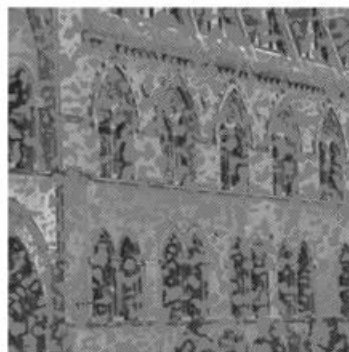
blur $\sigma = 1.0$ pixels



blur $\sigma = 1.5$ pixels



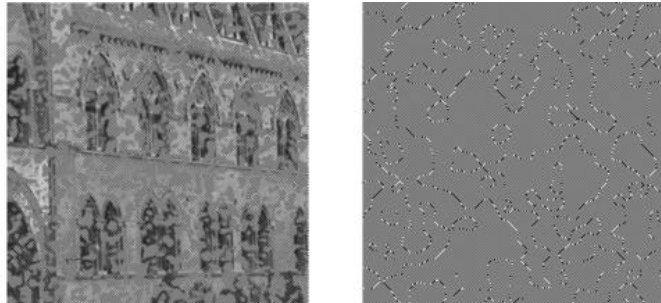
$\hat{f}(x,y)$



Inverse Filtering

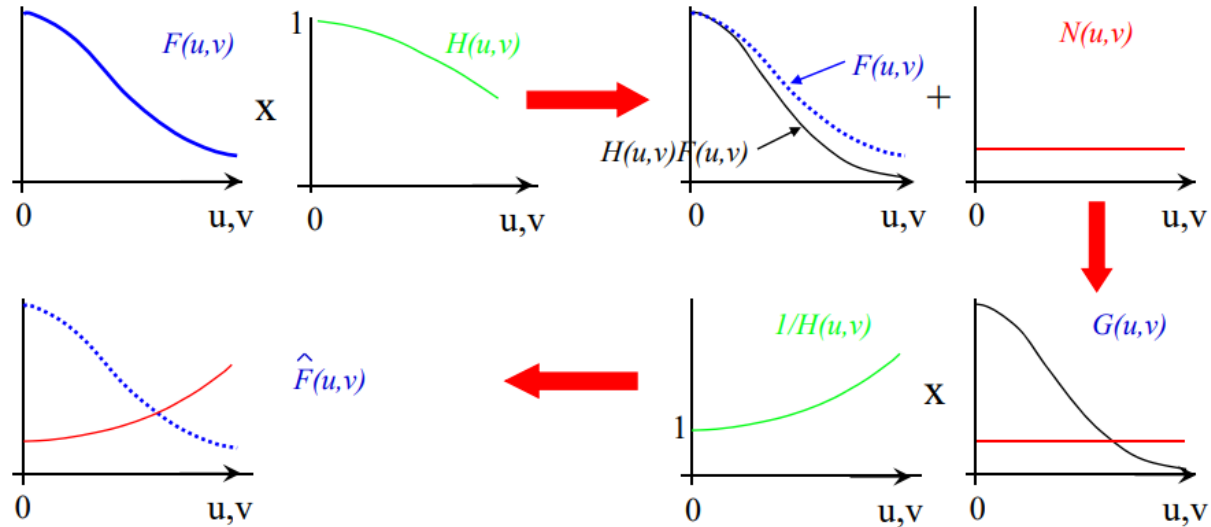
$$\hat{F}(u, v) = F(u, v) + \frac{N(u, v)}{H(u, v)}$$

- ▶ As you can see that if $H(u, v)$ has zero or small values, the second term will dominate the whole image – As we can see in the case in the last slide



Inverse Filtering

- This is the problem of noise amplification



The Wiener Filtering

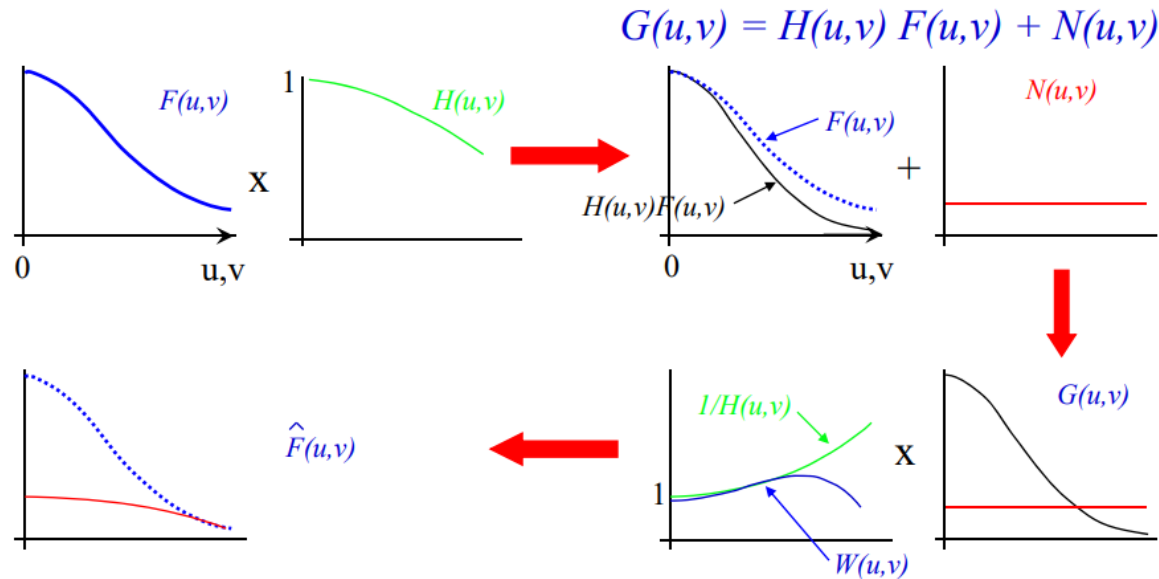
- ▶ Minimum Mean Square Error (Wiener) filter is a restoration filtering which can handle noise

$$\hat{F}(u, v) = W(u, v) G(u, v)$$

$$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + K(u, v)}$$

- ▶ Where the least square error of the image is $\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x, y) - \hat{f}(x, y)]^2$

The Wiener Filtering



The Wiener Filtering

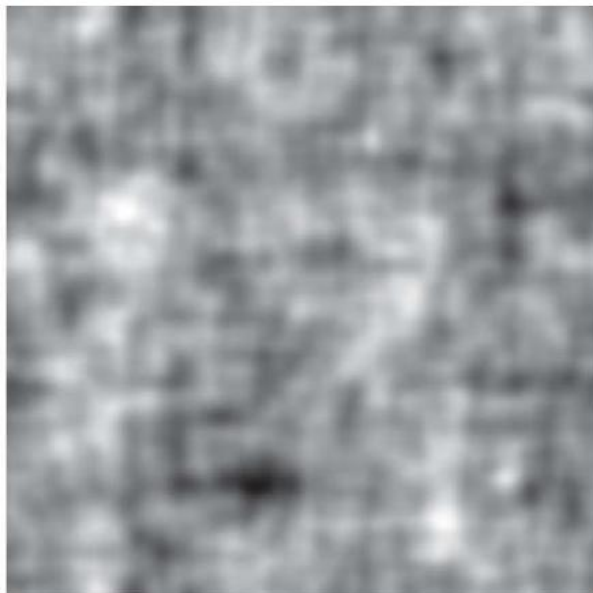


Original Image



Degraded image

The Wiener Filtering



Full inverse filtering



Wiener filter

The Wiener Filtering

blur $\sigma = 1.5$ pixels

noise $\sigma = 0.3$ grey levels

$$\hat{F}(u,v) = W(u,v) G(u,v) \quad W(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K(u,v)}$$

$g(x,y)$



$\hat{f}(x,y)$



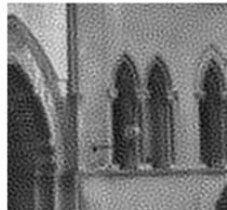
$K = 1.0 \times 10^{-5}$



$K = 1.0 \times 10^{-3}$



$K = 1.0 \times 10^{-1}$



Case study 1



Degradation model

$$g(x, y) = \frac{1}{T} \int_{-T/2}^{T/2} f(x - x_0(t), y) dt$$

Require $H(u, v)$ for Wiener filter

$$\begin{aligned}
 G(u, v) &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x, y) e^{-j2\pi(ux+vy)} dx dy, \\
 &= \frac{1}{T} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \left\{ \int_{-T/2}^{T/2} f(x - x_0(t), y) dt \right\} e^{-j2\pi(ux+vy)} dx dy
 \end{aligned}$$

interchange order of spatial and temporal integration

$$G(u, v) = \frac{1}{T} \int_{-T/2}^{T/2} \left\{ \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x - x_0(t), y) e^{-j2\pi(ux+vy)} dx dy \right\} dt$$

Fourier transform of $f(x - x_0(t), y)$.

$$\begin{aligned}
 G(u, v) &= \frac{1}{T} \int_{-T/2}^{T/2} F(u, v) e^{-j2\pi u x_0(t)} dt \\
 &= F(u, v) \frac{1}{T} \int_{-T/2}^{T/2} e^{-j2\pi u x_0(t)} dt \\
 &= F(u, v) H(u, v)
 \end{aligned}$$

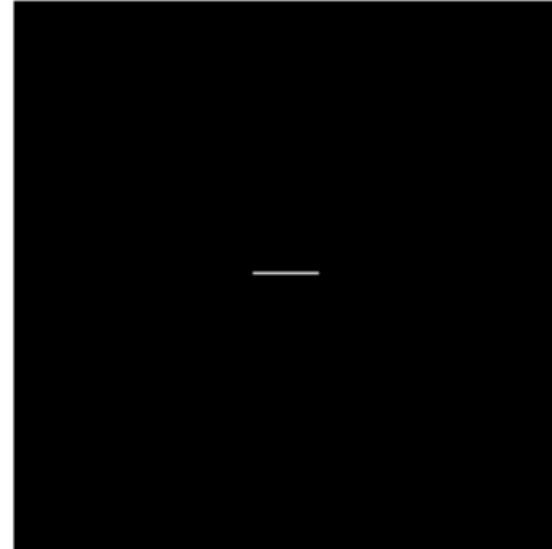
where

$$H(u, v) = \frac{1}{T} \int_{-T/2}^{T/2} e^{-j2\pi u x_0(t)} dt$$

suppose $x_0(t) = st$, and $sT = d$ pixels

FT of ... $h(x, y) = \text{hat}_d(x)\delta(y)$

$$\begin{aligned} H(u, v) &= \frac{1}{T} \int_{-T/2}^{T/2} e^{-j2\pi ust} dt \\ &= \frac{1}{T} \left[\frac{e^{-j2\pi ust}}{-j2\pi us} \right]_{-T/2}^{T/2} \\ &= \frac{1}{j2\pi ud} (e^{j\pi ud} - e^{-j\pi ud}) \\ &= \text{sinc}\pi ud \end{aligned}$$



Note, $H(u, v)$ has zeros – a problem for an inverse filter

Case study 1

blur = 20 pixels

$$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + K(u, v)}$$



1. Compute the FT of the blurred image
2. Multiply the FT by the Wiener filter $\hat{F}(u, v) = W(u, v) G(u, v)$
3. Compute the inverse FT

Case study 2

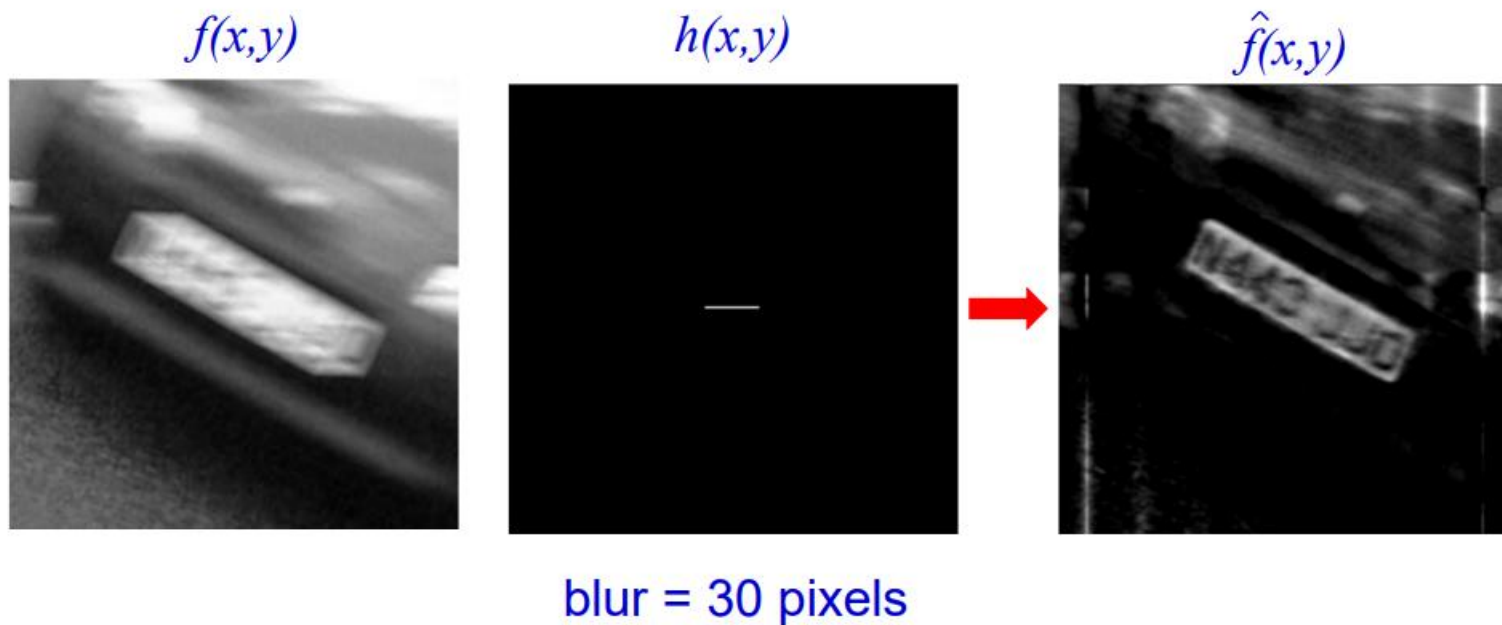




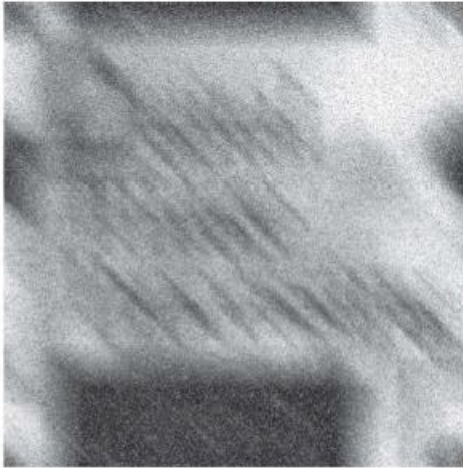
Algorithm

1. Rotate image so that blur is horizontal
2. Estimate length of blur
3. Construct a bar modelling the convolution
4. Compute and apply a Wiener filter
5. Optimize over values of K

Case study 2



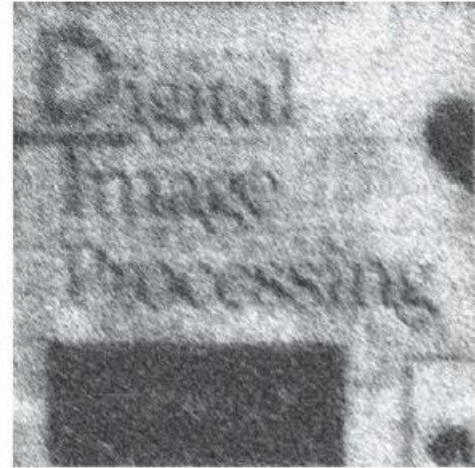
Case study 3



Degraded image



Full inverse filtering

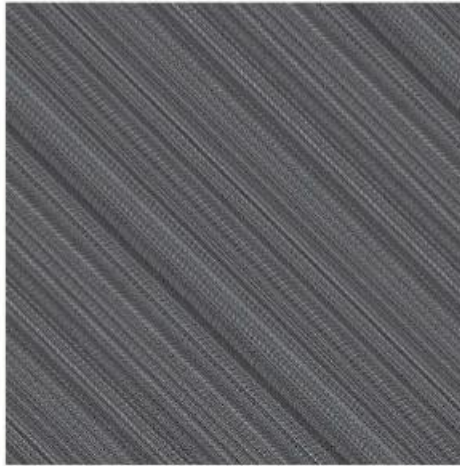


Wiener filter

Case study 3



Degraded image
With noise variance one
order of magnitude less



Full inverse filtering



Wiener filter

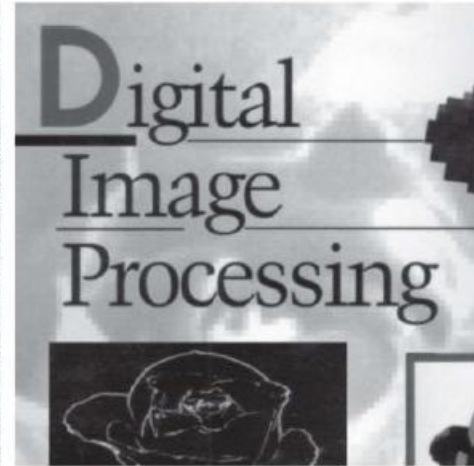
Case study 3



Degraded image
With noise variance five
orders of magnitude less



Full inverse filtering



Wiener filter



End of Week 6